

Vector Databases and Vector Embeddings-Review

1st Sanjay Kukreja*
Department of Machine Learning
SP Jain School of Global Management
 Mumbai, India
 *Corresponding email:
 sanjay.ds18dba008@spjain.org

2nd Tarun Kumar
COE AI-ML
eClerx Services Ltd.
 Chandigarh, India
 tk7ua1678@gmail.com

3rd Dr. Vishal Bharate
COE AI-ML
eClerx Services Ltd.
 Pune, India
 vishalbharate@gmail.com

4th Amit Purohit
COE AI-ML
eClerx Services Ltd.
 Mumbai, India
 amit.iitgn@gmail.com

5th Dr. Abhijit Dasgupta
Department of Data Science
SP Jain School of Global Management
 Mumbai, India
 abhijit.dasgupta@spjain.org

6th Dr. Debashis Guha
Department of Machine Learning
SP Jain School of Global Management
 Mumbai, India
 Debashis.guha@spjain.org

Abstract—This research paper aims to present a comprehensive survey of vector databases and vector embedding techniques. A concise overview of the evolution, architecture, advantages and challenges of vector databases are presented in this paper. To convert unstructured data into vectors, various embedding techniques with their in-depth technical description are also surveyed in this research paper. The existing vector databases' characteristics and features are described to select an appropriate vector database. The embedding and indexing techniques are thoroughly discussed to help the research community experiment with combining them to develop their application. The challenges of vector databases, like selecting distance metrics, dimensionality, data integrity, cost, etc. are presented. This information on vector database systems will help researchers to exhibit further advancements in their data science applications.

Keywords—Artificial Intelligence, Computer Vision, Embeddings, Generative AI, Large Language Model

I. INTRODUCTION

The evolution of Generative AI has grabbed the attention of the research community towards unstructured data. The storage and retrieval of unstructured data became critical with traditional databases. Vector databases play a crucial role in the retrieval and processing of unstructured data [1, 2] as there is increasing demand for large language model applications.

A vector database is a type of database to store and retrieve vector data. Vectors are the numerical representations of data. The unstructured data such as text, image, video and audio is converted into high dimensional vector data employing vector embedding techniques [3]. Vector embedding is the machine-learning technique which is responsible for this conversion. Understanding the interdependent relationship between vector databases and vector embeddings is helpful in advancing the capabilities of data-driven applications.

Mathematically, a vector database can be modeled and is given by Equation [1],

$$v = (i, n, m) \quad (1)$$

Where,

- v - Set of tuple of vectors
- i - Identifier
- n - dimension
- m - attribute of real values

The specialty of vector databases is that they are focused on the storage of vectors in a proficient manner [4, 6]. Also, vector databases are able to handle real-time high-dimensional vector indices. In vector databases, similar vectors are searched using a vector composed of a query called vector embeddings. Hence, vector databases are optimized for storing and utilizing vectors as outcomes received from large language models.

As the data is converted into high-dimensional vectors, efficient storage is also crucial. This storage is developed with the aid of vector indexing techniques. Various vector indexing techniques are discussed in detail in this research paper. The existing applications can also find tremendous change due to upcoming variations in the technology with vector databases.

The purpose of this research paper is to provide a comprehensive survey of existing vector databases, researching their design principles and indexing techniques. Also, another purpose is to examine the landscape of vector embeddings, separating the various techniques employed for effective data representation.

The key contributions are as follows:

1. Numerous features of different vector databases are collected and presented.
2. Different vector embeddings are presented and discussed in detail.
3. Variety of indexing techniques are surveyed and presented for ready reference.
4. Collected information about challenges associated with vector databases.

This paper is further organized into the following parts. Section II represents related work of vector database systems. A thorough literature survey regarding existing vector databases is discussed in it. The features of selected vector databases are presented in it for ready reference by researchers. Section III discusses vector embeddings. The techniques of vector embeddings are presented and discussed in this section. The detailed architecture is presented and discussed in section IV. Section V highlights the various indexing techniques utilized in vector databases. The evaluation metrics and benchmarks are discussed in section VI. Section VII gives a brief overview of the advantages, applications and use cases of the vector databases. Lastly, a

thorough discussion of challenges and future trends associated with vector databases is presented in section VIII. The research paper is concluded by summarizing the key contributions and findings of this literature survey.

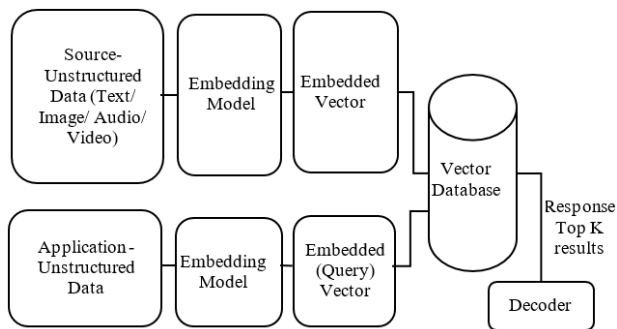


Fig. 1. Block Schematic of Storing and Retrieving Data in Vector Database

II. RELATED WORK-VECTOR DATABASE SYSTEMS

The task of a vector database system is to store the unstructured data into vectors and retrieve similar vectors when the database is queried [5, 8]. This includes vector embedding and vector indexing techniques. The process of storing and retrieving data is presented in Figure 1.

The source data in unstructured format, such as image, text, audio, and video, is fed to the system to create a vector database. The source unstructured data is split into smaller chunks. These smaller chunks of unstructured data are converted into embedded vectors by means of an embedding model. The converted output, i.e. embedded vector, is stored in the vector store. The vector database indexing represents the positions of these chunks in the multidimensional vector space.

Various available databases are categorized based on self-hosted or cloud-based and serverless or client-server-based. This categorization is shown in Figure 2. It can be seen from Figure 2 that some databases support both self-hosted as well as on- cloud hosting. Also, few of these databases support serverless or client-server properties. Based on constraints and the need for an application, one can choose a vector database system to be used in their application.

The features and characteristics of selected databases are presented in Table 1. Various parameters, like technical and non-technical, appropriate for vector databases can be seen in the table. These features will be helpful for data science enthusiasts in the selection of appropriate vector databases based on the need of the application.

III. EMBEDDINGS

In order to represent any kind of data, such as text, image, audio, or video, into semantically meaningful clusters, embeddings are used. Embeddings convert the unstructured data into meaningful, semantic, multidimensional vector space. In machine learning, embedding is a way of representing data as points in n-dimensional space so that similar data points are clustered together. The various types of embeddings are shown in Figure 3. These types are explained in detail as follows.

A. Word embeddings

In word embeddings [19], the individual word is presented as a vector. Various techniques like Word2Vec, FastText and GloVe are used to extract semantic relationships and

contextual information from large text corpus using word embeddings. These techniques are explained as follows.

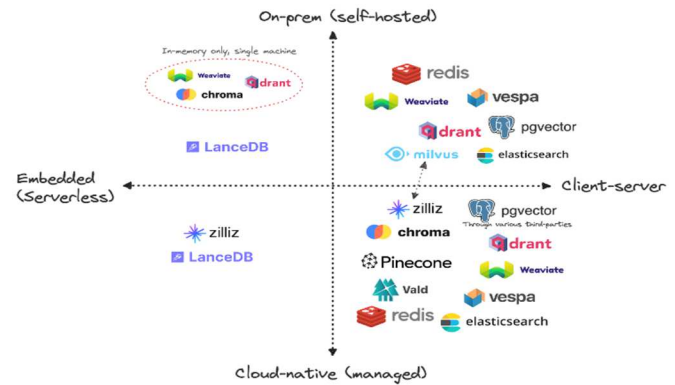


Fig. 2. Combinations of Different Vector Databases [54]

TABLE I. FEATURES OF VECTOR DATABASES

Database	Chroma [16]	Weaviate [12]	Qdrant [14]	Vespa [17]	Milvus [7, 15]
Feature					
Link	[16]	[12]	[14]	[17]	[7]
Release Date	Oct 23, 2022	May 21, 2019	Apr 6, 2021	Sep 22, 2021	Oct 21, 2019
Github Star (18/10/2023)	9k	7.8 k	13.3k	4.9k	23.5k
Open Source	Yes	Yes	Yes	Yes	Yes
Metric Types	Cosine	Cosine	Inner Product, Cosine Similarity, Euclidean (L2)	Euclidean, Angular, Inner Product, Geo Degree, Hamming	L2, Inner Product, Hamming, Jaccard, Tanimoto, superstructure, Substructure
By default embedding model used	all-MiniLM-L6-v2	-	BAAI/bge-small-en-v1.5	built-in embedder-Bert and HuggingFace	-
Programming Language	Python/TypeScript	Python, Go	Python, Rust	Java, Python	C++, Python
Query Language	SQL, JSON	GraphQL, JSON	Python API, JSON	SQL	SQL
Distributed	Yes	Yes	Yes	Yes	Yes
Indexing Methods	HNSW	HNSW (PQ), customized HNSW	Customized HNSW	HNSW, BM25	HNSW, Annoy, IVF_PQ, IVF_SQ8
Query Types	Range search, Nearest Neighbor search	Range search, Nearest Neighbor search, semantic search	Range search, Nearest Neighbor search	Range search, Nearest Neighbor search, semantic search	Range search, Nearest Neighbor search, semantic search
Scalability	-	Horizontal	Horizontal	Horizontal	Horizontal
Storage Backend	Clickhouse	-	-	Custom	RocksDB, MinIO
License	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0	AGPLv3

- **Word2Vec**- It is a technique to convert words into vectors. Skip Gram and Common Bag of Words (CBOW) are the techniques to achieve vectorization of the word.
- **FastText**- It is an extension of word2vec. It is a word embedding technique that provides embedding to the character n-grams.
- **GloVe**- It is a type of word embedding that encodes the co-occurrence probability ratio between two words as vector differences.

B. Sentence embeddings

In sentence embedding [20], the complete sentence is represented as a vector. In order to capture the context and complete meaning of the sentence, techniques like Universal Sentence Encoder (USE) and SkipThought are used to generate the vectors. These are explained as follows.

- **Universal Sentence Encoder (USE)**- The embeddings produced by the Universal Sentence Encoder are approximately normalized.
- **SkipThought**- It can be used where there is continuity across sentences.

C. Document embeddings

In document embeddings [21], entire documents are represented as vectors. The entire context is captured by means of techniques like Doc2Vec. Paragraph Vectors are designed to learn document embeddings. It is explained as follows.

- **Doc2Vec**- It is a widespread technique in Natural Language Processing that enables the representation of documents as vectors.

D. Image embeddings

In image embeddings [22], images are represented as vectors. The image features are represented by means of these vectors using techniques like convolutional neural networks (CNNs) and pre-trained models like VGG and ResNet. These can be used for tasks like image classification, image similarity, and object detection.

- **VGG** – It is a pre-trained model and a type of CNN Architecture.
- **ResNet**- It is the idea that every additional layer should more easily contain the identity function.

E. User embeddings

In user embeddings [23], the platform or whole system is represented as vectors. The task of user embeddings is to retrieve the user behaviors, characteristics and preferences. These are used in different applications, from recommendation to personalized marketing as well as user segmentation, in recommendation systems.

F. Product embeddings

In product embeddings [24], products in e-commerce or recommendation systems are represented as vectors. The task of product embedding is to capture product features, their attributes and semantic information if applicable. Using these embeddings, users can develop applications like recommendations.

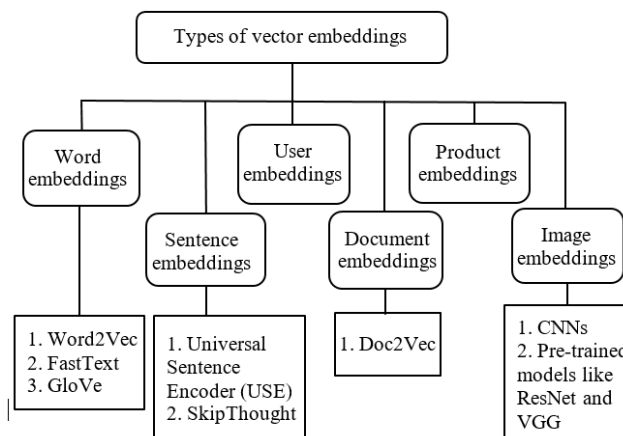


Fig. 3. Vector Embedding Types

G. Multi-modal embeddings

As there is an increase in the use of Generative AI, the demand for multimodal embeddings [27] has also increased. To generate images from the input text, multimodal types of embeddings are used. The CLIP, VisualBert, and LXMERT are some of the examples of multimodal embeddings. Also, multi2vec-clip- is used in the Weaviate vector database.

IV. VECTOR DATABASE ARCHITECTURE

The different vector database systems surveyed have not provided system architecture. Figure 4 shows the overall architecture of the Milvus vector database. In general, it consists of a client interface, query processor, vector storage, and indexing module.

The interaction of the database with the application is performed by means of a client interface. The query sent by an application is processed under the query processor. It searches for the k similar vectors and provides the result. Efficient storage is developed with the help of an indexing module.

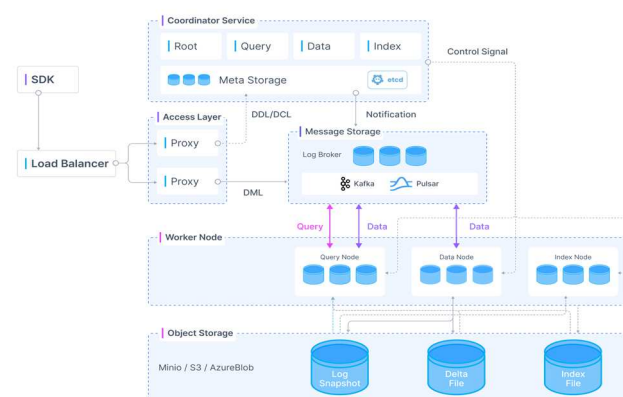


Fig. 4. Overall Architecture of Milvus [15]

V. INDEXING TECHNIQUES

Indexing techniques are helpful to structure and organize vector data efficiently. Due to the increase in dimensionality of vector data (high-dimensionality), traditional approaches like hashing and B-tree are unsuitable for indexing. Hence, special techniques like ball-trees, R-tree (space partitioning method), and k-d trees [28] are developed. The working of

these techniques reduces the search speed by dividing vector space into regions.

Figure 5 shows various indexing techniques for search in vector databases. They broadly classified into a nearest neighbor and approximate nearest neighbor. A few of these techniques are described as follows.

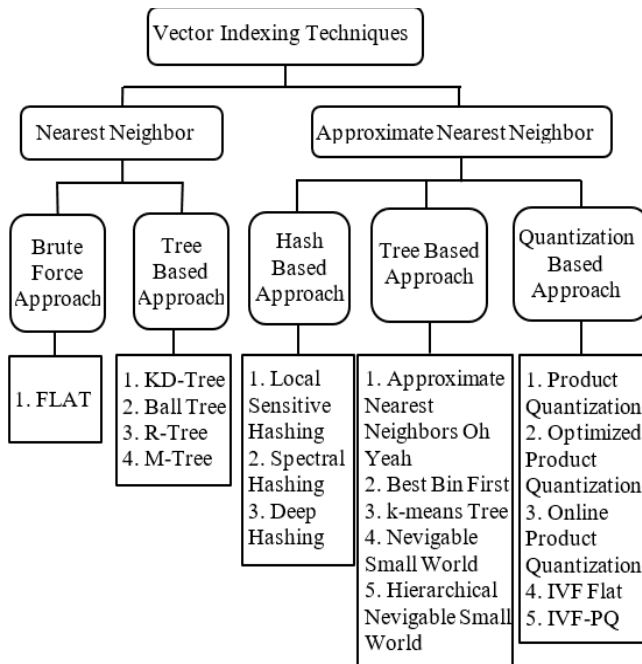


Fig. 5. Vector Indexing Techniques for Search in Vector Databases

- Flat Index: It is a type of brute-force search [48]. As the data is not organized but all the vectors in the dataset are scanned and the search is completed.
- Inverted File (IVF) [49]: Vector space gets divided into a number of clusters. Each cluster contains information about vectors in the form of an inverted file.
- IVF Flat: This is a combination of IVF and flat index. It uses the IVF index to partition the data into clusters and then uses the flat index (brute-force search) within each cluster.
- Hierarchical Navigable Small World (HNSW) [44]: HNSW builds a multi-layer navigation graph to represent the vector space.
- Product Quantization (PQ) [45]: PQ index compresses vectors into compact codes and is beneficial for large-scale, high-dimensional data.
- IVF-PQ [50]: It is a combination of IVF and PQ. It uses IVF to partition the data, and within each cluster, it applies PQ for further compression.
- Binary Inverted Index: The binary, inverted index is used for binary vectors, which partitions binary vectors into buckets based on the Hamming distance.
- k-d tree - A k-d tree is a data structure for organizing points in a k-dimensional space. The k-d tree is a binary tree in which every node is a k-dimensional point.
- R-Tree [32] - An index organizes access to data so that entries can be found quickly without searching every

row. The R-tree access method enables you to index multidimensional objects. Queries that use an index execute more quickly and provide a significant performance improvement.

- Ball tree [29,30,31] - The ball tree nearest-neighbor algorithm examines nodes in depth-first order, starting at the root.
- M-Tree [33] - An M-Tree (Metric Tree) is a data structure designed for organizing and searching large datasets in metric spaces. It enables efficient similarity search and nearest neighbor queries by leveraging the properties of metric spaces.
- Locality Sensitive Hashing (LSH) indexes [34] - Locality Sensitive Hashing is an indexing strategy that optimizes for speed and finding an approximate nearest neighbor, instead of doing an exhaustive search to find the actual nearest neighbor as is done with flat indexing.
- Spectral hashing [35]- Spectral hashing (SpH) is an efficient and simple binary hashing method, which assumes that data are sampled from a multidimensional uniform distribution.
- Deep hashing [36]- The Deep hashing method is widely applied in the field of image retrieval because of its advantages of low storage consumption and fast retrieval speed.
- Approximate Nearest Neighbors Oh Yeah [37]- Annoy Index works by constructing a tree-based index structure for quick searches in high-dimensional spaces.
- Best Bin First [38,39]- Best bin first is an approximate algorithm which returns the nearest neighbor for a large fraction of queries and a very close neighbor otherwise.
- Navigable Small World [41,42,43]- A graph-based algorithm designed for finding approximate nearest neighbors within a dataset. In this algorithm, the data points are represented as nodes in a graph, and each node is connected to a fixed set of nearby nodes.
- Optimized Product Quantization [46] - It aims at finding an optimal space decomposition of PQ, which introduces two different solutions.
- Online Product Quantization [47] - It is a variation of product quantization (PQ), which is a technique for compressing high-dimensional vectors into smaller and more efficient representations.

VI. EVALUATION METRICS AND BENCHMARKS

The performance of vector databases is evaluated on the standard datasets. Standard evaluation metrics are used to validate the performance of the vector database. The various performance metrics for the evaluation of vector databases are shown in Figure 6. These metrics are discussed briefly below.

- Upload Time- It is the time required to upload the entire dataset.

- **Index (Build) Time**- It is the time required to index the entire dataset.
- **Throughput**- It is the rate at which a database can process and handle a specific volume of queries (Queries per second) in a second.
- **Latency**- It is the time taken to return the results to a given request.
- **Precision**- It is defined as a Total number of documents retrieved that are relevant/Total number of documents that are retrieved.
- **Recall**- It is defined as a Total number of documents retrieved that are relevant/Total number of relevant documents (percentage of relevant results returned by query).
- **Retrieval Speed** – It is the time to retrieve and return the results in response to a query.

VII. ADVANTAGES

There are various advantages when we use vector databases. The unstructured data in computer vision, natural language processing, audio and video applications will find an exciting change in performance. The various advantages are listed and discussed briefly as follows.

- **High-dimensionality search**: The most significant advantage of vector databases is the multi/high dimensionality search of vectors. In order to manage applications in the domain of image processing, natural language processing high-dimensionality search plays a crucial role.
- **Scalability**: Vector databases scale horizontally. They are capable of efficiently storing and retrieving large amounts of vector data. The real-time search and retrieval of huge amounts of data is managed with vector databases.
- **Flexibility**: Vector databases can handle various vector data types, such as sparse and dense vectors. They have the capability to handle multiple data types like numerical, text, and binary data.
- **Performance**: It provides faster search as compared to traditional databases.
- **Customizable indexing**: It allows custom indexing schemes for specific use cases and data types.

VIII. CHALLENGES AND FUTURE TRENDS

From the available literature, it is found that there are specific challenges associated with vector databases. These challenges are discussed in detail as follows.

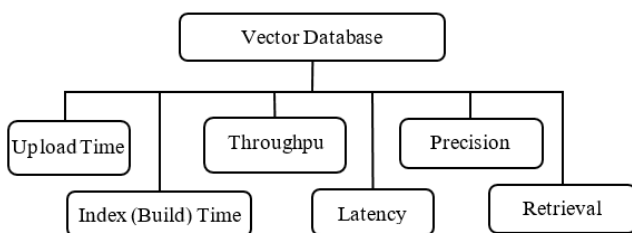


Fig. 6. Performance Metrics to Benchmark Vector Database

- **Increase of dimensionality**: The computational requirements as the sparsity of the data increases with the increase of dimensionality. This significant hurdle has to be handled, and a golden mean should be achieved to mitigate the issue of an increase in dimensionality.
- **Choice of distance metrics**: Selecting suitable distance metrics for vector similarity search is critical but challenging. In order to get appropriate similarity, search results and accuracy with various data types may require specific distance metrics.
- **Consistency and data integrity**: When working with real-time applications of vector databases, it is difficult to assure consistency and data integrity. While designing and implementing the system, these issues need to be considered.
- **Indexing and query optimization**: This is one of the open research problems. In order to balance the trade-off between indexing, storage and execution of query, it is necessary to achieve the optimum performance from vector databases.
- **Cost optimization**: The storage requirements of vector databases are more as compared to traditional databases. In order to tackle this issue, the designer has to choose the right combination of indexing and search algorithms along with the selection of storage devices.
- **Automatic tuning and configuration**: The actual performance of the system can vary based on the selection of data features and algorithm parameters. The manual methods to select appropriate algorithms are not efficient due to high dimensionality of the vector data. The specific index structure is not supported by all the vector databases. Hence, there is a scope to optimize it by adopting some machine learning techniques.

IX. CONCLUSION

The vector databases are surveyed in detail and presented in this research paper. The basic working of a vector database with the techniques of indexing, vector embeddings, search is discussed in detail. The combination of these techniques may OPTIMIZE the system's performance. The open research problems where researchers can further work are also listed. In order to select a specific vector database, the characteristics and performance evaluation metrics are also presented, which will act as a ready reference for practitioners.

In future researchers can work to optimize the cost of the vector databases. Also choice of embeddings, indexing and dimensionality can also be further researched. There is scope to optimize the performance of the vector databases using combination of various methods enlisted in this paper.

REFERENCES

- [1] Samrat Sahoo, Nitin Paul, et.al., "The Universal NFT Vector Database: A Scaleable Vector Database for NFT Similarity Matching", Proceedings of the VLDB Endowment, Vol. 17, No. 1, arXiv:2303.12998.
- [2] Toni Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges", arXiv:2309.11322,2023.
- [3] GUILLAUME TOUYA and IMRAN LOKHAT, "Deep Learning for Enrichment of Vector Spatial Databases: Application to Highway Interchange", ACM Transactions on Spatial Algorithms and Systems, Vol. 6, No. 3, Article 21, pp 1–21, April 2020.

- [4] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, April 2020.
- [5] Rentong Guo, Xiaofan Luan, et.al., "Manu: A Cloud Native Vector Database Management System", *Proceedings of the VLDB Endowment*, Vol. 15, No. 12.
- [6] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis, "The case for learned index structures", *Proceedings of the 2018 International Conference on Management of Data*, ACM, 2018.
- [7] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, C. Xie, "Milvus: A purpose-built vector data management system", *Proceedings of the 2021 International Conference on Management of Data*, ACM, 2021.
- [8] J. Johnson, M. Douze, H. Jegou, "Billion-scale similarity search with GPUs", *IEEE Transactions on Big Data* 7, 2021, 535-547.
- [9] <http://github.com/spotify/annoy>.
- [10] <http://elastic.co>.
- [11] <http://vald.vdaas.org>.
- [12] <http://weaviate.io>.
- [13] <http://pinecone.io>.
- [14] <http://qdrant.tech>.
- [15] <http://milvus.io>.
- [16] <http://trychroma.com>.
- [17] <http://vespa.ai>.
- [18] <https://github.com/lancedb/lancedb>.
- [19] Kusner, M., Sun, Y., Kolkin, N. & Weinberger, K., "From Word Embeddings to Document Distances", *Proceedings of the 32nd International Conference on Machine Learning*, in *Proceedings of Machine Learning Research* 37:957-966, 2015.
- [20] Q. Chen, Y. Peng and Z. Lu, "BioSentVec: creating sentence embeddings for biomedical texts", *IEEE International Conference on Healthcare Informatics (ICHI)*, Xi'an, China, 2019, pp. 1-5.
- [21] R.A. Sinoara, J. Camacho-Collados, R.G. Rossi, R. Navigli, S.O. Rezende, "Knowledge-enhanced document embeddings for text classification", *Knowl. Base Syst.*, 163, 2019, pp. 955-971.
- [22] B. Barz and J. Denzler, "Hierarchy-Based Image Embeddings for Semantic Image Retrieval", *IEEE Winter Conference on Applications of Computer Vision (WACV)*, Waikoloa, HI, USA, 2019, pp. 638-647.
- [23] Casper Hansen, Christian Hansen, Lucas Maystre, Rishabh Mehrotra, Brian Brost, Federico Tomasi, and Mounia Lalmas, "Contextual and Sequential User Embeddings for Large-Scale Music Recommendation", *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys '20)*, Association for Computing Machinery, New York, NY, USA, 2020, 53-62.
- [24] Flavian Vasile, Elena Smirnova, and Alexis Conneau, "Meta-Prod2Vec: Product Embeddings Using Side-Information for Recommendation", *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*, Association for Computing Machinery, New York, NY, USA, 2016, 225-232.
- [25] A. L. Cramer, H. -H. Wu, J. Salamon and J. P. Bello, Look, "Listen, and Learn More: Design Choices for Deep Audio Embeddings", *ICASSP 2019 - IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, UK, 2019, pp. 3852-3856.
- [26] S. -H. Hu, Yikang Li and Baixin Li, "Video2vec: Learning semantic spatio-temporal embeddings for video representation", *23rd International Conference on Pattern Recognition (ICPR)*, Cancun, 2016, pp. 811-816.
- [27] Limeng Cui, Suhang Wang, and Dongwon Lee, "SAME: sentiment-aware multi-modal embedding for detecting fake news", In *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM '19)*, Association for Computing Machinery, New York, NY, USA, 2020, 41-48.
- [28] J. L. Bentley, "Multidimensional binary search trees used for associative searching", *Communications of the ACM*, vol. 18, no. 9, p. 509-517, 1975.
- [29] S. M. Omohundro, "Five balltree construction algorithms", *Berkeley: International Computer Science Institute*, 1989.
- [30] T. Liu, A. W. Moore, A. Gray, and K. Yang, "New algorithms for efficient high-dimensional nonparametric classification", *Journal of machine learning research*, vol. 7, no. 6, 2006.
- [31] S. Kumar and S. Kumar, "Ball*-tree: efficient spatial indexing for constrained nearest-neighbor search in metric spaces", *arXiv preprint arXiv:1511.00628*, 2015.
- [32] A. Guttman, "R-trees: a dynamic index structure for spatial searching", in *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*, p. 47-57, 1984.
- [33] P. Ciaccia, M. Patella, and P. Zezula, "M-tree: An efficient access method for similarity search in metric spaces", in *Vldb*, vol. 97, p. 426-435, 1997.
- [34] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions", in *Proceedings of the twentieth annual symposium on Computational geometry*, p. 253-262, 2004.
- [35] Y. Weiss, A. Torralba, and R. Fergus, "Spectral hashing", *Advances in neural information processing systems*, vol. 21, 2008.
- [36] H. Liu, R. Wang, S. Shan, and X. Chen, "Deep supervised hashing for fast image retrieval", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, p. 2064-2072, 2016.
- [37] B. E and S. AB., "Annoy (approximate nearest neighbors oh yeah)". [DB/OL]. (2015) [2023-10-17]. 3, 2015.
- [38] B. J. S and L. D. G., "Shape indexing using approximate nearest neighbour search in high-dimensional spaces", *Proceedings of IEEE computer society conference on computer vision and pattern recognition*, pp. 1000-1006, 1997.
- [39] H. Liu, M. Deng, and C. Xiao, "An improved best bin first algorithm for fast image registration", *Proceedings of 2011 International Conference on Electronic & Mechanical Engineering and Information Technology*, vol. 1, pp. 355-358, 2011.
- [40] T. P, T. P, and S. M., "K-means tree: an optimal clustering tree for unsupervised learning", *The Journal of Supercomputing*, vol. 77, pp. 5239-5266, 2021.
- [41] P. A, M. Y, L. A, and K. A., "Approximate nearest neighbor search small world approach", *International Conference on Information and Communication Technologies & Applications*, 2011.
- [42] M. Y, P. A, L. A, and K. A., "Scalable distributed algorithm for approximate nearest neighbor search problem in high dimensional general metric spaces", *Similarity Search and Applications: 5th International Conference, SISAP 2012, Toronto, ON, Canada, August 9-10, 2012. Proceedings 5*, pp. 132-147, 2012.
- [43] M. Y, P. A, L. A, and K. A., "Approximate nearest neighbor algorithm based on navigable small world graphs", *Information Systems*, vol. 45, pp. 61-68, 2014.
- [44] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs", *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, p. 824-836, 2018.
- [45] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search", *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 1, p. 117-128, 2010.
- [46] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization", *IEEE transactions on pattern analysis and machine intelligence*, vol. 36, no. 4, p. 744-755, 2013.
- [47] D. Xu, I. W. Tsang, Y. Zhang, and J. Yang, "Online product quantization", *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 11, p. 2185-2198, 2018.
- [48] B. A. Trakhtenbrot, "A Survey of Russian Approaches to Perebor (Brute-Force Searches) Algorithms", *Annals of the History of Computing*, vol. 6, no. 4, pp. 384-400, Oct.-Dec. 1984.
- [49] Kazuo Aoyama, Kazumi Saito, Tetsuo Ikeda, "Inverted-File k-Means Clustering: Performance Analysis", *arXiv:2002.09094*.
- [50] James O' Neill and Sourav Dutta, "Improved Vector Quantization for Dense Retrieval with Contrastive Distillation", *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*, Association for Computing Machinery, New York, NY, USA, 2072-2076, 2023.
- [51] Aggarwal, C.C., Hinneburg, A., Keim, "D.A.: On the surprising behavior of distance metrics in high dimensional space", *ICDT (2001)*.
- [52] <https://thedataquarry.com/posts/vector-db-4/>.